

Computer Vision in Mechanical Engineering

A beginner-friendly tutorial on image datasets, convolutional neural networks, transfer learning, detection, segmentation, and inspector-ready visualization

Prepared for Mechanical Engineering students with beginner-level Python experience

April 2026

Audience	Mechanical Engineering students who are new to computer vision, deep learning for images, and engineering image interpretation.
Major themes	Part I: image datasets and convolutional neural networks. Part II: detection, segmentation, and visual communication for inspection.
Mechanical focus	Crack and defect classification, surface defect localization, corrosion mapping, mechanical component and wind tunnel test section inspection imagery, annotated field reporting, and safety-aware evaluation.

Teaching philosophy. The tutorial begins with intuition, then builds the minimum Python and image vocabulary, then explains how convolutional neural networks learn from images, and finally moves to detection, segmentation, and report-ready visual outputs. Throughout the document, Mechanical Engineering interpretation comes first: the goal is not only to train a model, but also to produce evidence that an inspector or asset manager can trust.

Table of Contents

- 1 Detailed tutorial outline and improved teaching sequence
- 2 Introduction to computer vision in Mechanical Engineering
- 3 Python and image-learning mounting bases needed for this tutorial

- 4 Deep learning mounting bases for image analysis
- 5 Lecture Part I: Image datasets and convolutional neural networks
 - 5.1 Image data preparation
 - 5.2 Transfer learning
 - 5.3 Classifier evaluation in safety-critical Mechanical Engineering contexts
 - 5.4 Theory and mathematics for image classification
- 6 Lecture Part II: Detection and visualization
 - 6.1 Computer vision task types
 - 6.2 Simple detection and segmentation workflows
 - 6.3 Evaluation ideas for detection and segmentation
 - 6.4 Inspector-friendly visual outputs
- 7 Mechanical Engineering mini-cases and a practical workflow checklist
- 8 Conclusion
- References

1 Detailed tutorial outline and improved teaching sequence

Before the full tutorial begins, it is helpful to show the teaching sequence explicitly. Beginners often struggle when they see a complete image-classification or detection script before they understand what an image tensor is, what a label means, or why a confusion matrix matters. The sequence below is designed to move from the simplest concepts toward real Mechanical Engineering workflows.

1. Start with the engineering problem: why photographs, drone images, and inspection frames are important sources of evidence in Mechanical Engineering practice.
2. Build a small shared vocabulary: arrays, image dimensions, channels, labels, probabilities, and the training-validation-test split.
3. Explain convolutional neural networks in intuitive terms before introducing code, so that filters, feature maps, activations, and pooling have clear physical meaning.
4. Show how to prepare image datasets, organize class folders, resize and normalize images, and apply augmentation when data are limited.
5. Train a lightweight baseline model first, then introduce transfer learning so students can see why pre-trained models are powerful on small defect datasets.
6. Evaluate models in safety-critical terms, especially with confusion matrices, precision, recall, F1-score, class imbalance, and false negatives.
7. Distinguish carefully between classification, detection, semantic segmentation, and instance segmentation, because each answers a different engineering question.
8. Finish with inspector-friendly visual outputs so that the model result becomes a clear, annotated engineering communication product rather than an abstract score.

Why this sequence works. A Mechanical Engineering student does not need every advanced deep-learning detail on day one. What is needed first is a reliable conceptual ladder: image data, model basics, safe evaluation, and practical visualization. That ladder makes later topics such as object detection and segmentation easier to understand.

Stage	Main question	Typical Mechanical Engineering example
Mounting bases	What do pixels, channels, labels, and probabilities mean?	Cracked versus intact surface images, turbine blade surface photographs, wind tunnel test section lining inspection frames
Part I	How do we build and evaluate an image classifier?	Material crack classification, corrosion-category prediction, asset condition ranking
Part II	How do we localize or segment damage and show it clearly?	Surface defect boxes, corrosion masks, spall highlighting, report-ready annotated photographs
Integration	How does the output support a human decision?	Inspector review, maintenance prioritization, safety screening, documentation and reporting

2 Introduction to computer vision in Mechanical Engineering

Computer vision is the branch of artificial intelligence that teaches computers to interpret images and video by learning patterns from pixel values [9], [10]. In Mechanical Engineering, images are not just illustrations. They are measurements with visual evidence. A photograph of a mechanical component, a wind tunnel test section lining, a machined surface, or a material slab can reveal cracks, corrosion staining, surface defects, spalling, surface wear, misalignment, or water ingress. When these visual signals are organized into datasets, a computer model can learn to support inspection, screening, and reporting.

High-resolution digital photography has already become an important inspection aid in mechanical component engineering because it supports visual documentation, crack and spall identification, and even geometry reconstruction workflows [1]. This makes Mechanical Engineering a natural home for computer vision. The subject matters because many inspections produce large image collections that are expensive to review manually, yet the consequences of missing a serious defect can be significant.

- Inspection and maintenance: classify whether a photographed surface is cracked, corroded, intact, wet, or deteriorated.
- Safety screening: flag images that require urgent human review, such as severe cracking or large surface defects.
- Localization: draw a box around a surface defect, damaged joint, exposed reinforcement region, or fallen material fragment.
- Segmentation: color the exact corrosion region or damaged material area pixel by pixel.
- Reporting: place the original photo, the annotated image, the model confidence, and short notes into a consistent report layout for inspectors.

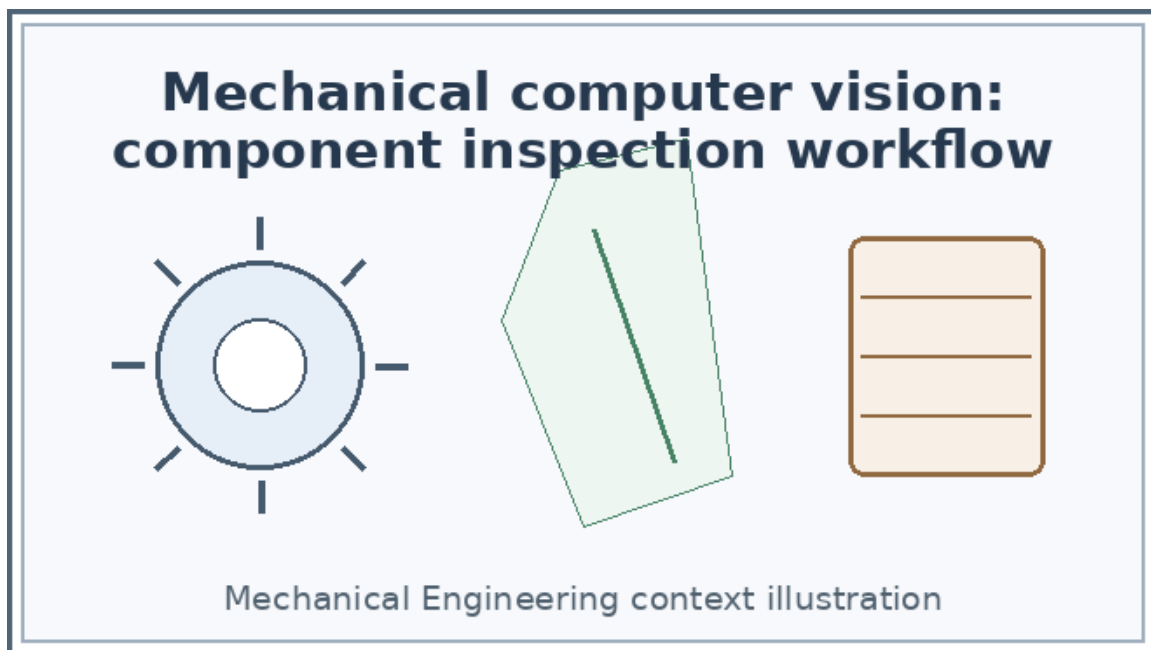


Figure 1. High-resolution mechanical component-inspection photography. The photograph illustrates why image interpretation matters in Mechanical Engineering: cracks, surface condition, and structural details can be reviewed visually and archived for later comparison. Source: [1].

Figure 1 shows the kind of inspection image that motivates engineering computer vision [1]. A beginner should immediately notice that the image is rich in structural context: there are material textures, shadows, water, geometry, and possible deterioration clues. A useful model must learn signal without becoming confused by irrelevant variation such as lighting or background scenery.

2.1 How computer vision relates to artificial intelligence, machine learning, deep learning, and convolutional neural networks

Artificial intelligence is the broadest term. It refers to systems that perform tasks normally associated with human reasoning or perception. Machine learning is a subset of artificial intelligence in which the system learns patterns from data rather than being hand-programmed for every rule [9], [10]. Deep learning is a subset of machine learning that uses many-layer neural networks to learn complex patterns directly from data. Convolutional neural networks, usually abbreviated as CNNs, are a family of deep-learning models designed especially for image data.

This relationship matters because a Mechanical Engineering student may hear all of these terms used almost interchangeably. The correct hierarchy is: artificial intelligence is broad, machine learning is narrower, deep learning is narrower still, and convolutional neural networks are one important deep-learning tool for image analysis [9], [10].

2.2 What kinds of Mechanical Engineering problems can be solved with computer vision

The form of the problem depends on the question being asked. If the question is 'Does this image contain cracking?', then classification is appropriate. If the question is 'Where is the surface defect?', then object detection is more suitable. If the question is 'Which pixels belong to corrosion or spalling?', then segmentation is the correct task definition [5].

- Crack detection in material, composite material, or coated steel surfaces
- Surface condition classification, such as intact, minor damage, or major damage
- Surface defect localization on machined surface photographs or production line-maintenance images
- Corrosion or spall region highlighting in mechanical component and wind tunnel test section images
- Asset condition assessment from routine inspection photography
- Annotated visual evidence for maintenance records and inspection reporting

Section summary. Computer vision gives Mechanical Engineers a way to turn visual inspection data into structured evidence. The technology sits inside the larger family of artificial intelligence and machine learning, but its usefulness depends on choosing the correct task definition and keeping engineering consequences in view.

3 Python and image-learning mounting bases needed for this tutorial

Only a small amount of Python is needed to understand the examples in this tutorial. The goal is not to become an expert software developer. The goal is to be able to read, reason about, and adapt beginner-friendly image-learning code.

3.1 Variables, arrays, folders, functions, classes, methods, and arguments

A variable is a name attached to a value. An array is a structured collection of numbers. In image work, arrays store pixel values. A folder is simply a way to group images into categories or datasets. A function performs a reusable task. A class is a blueprint for constructing an object, and a method is a function that belongs to an object. An argument is the information passed into a function or method when it is called.

The short example below introduces variables, arrays, and basic inspection of shape and data type.

```
import numpy as np

# A grayscale image can be stored as a 2-D array
gray_image = np.random.randint(0, 256, size=(4, 5), dtype=np.uint8)

# A color image often uses three channels: red, green, and blue
rgb_image = np.random.randint(0, 256, size=(4, 5, 3), dtype=np.uint8)

print("Gray shape:", gray_image.shape)
print("RGB shape:", rgb_image.shape)
print("RGB data type:", rgb_image.dtype)
```

In practice, real inspection images are much larger than 4 by 5 pixels, but the same idea applies. The shape tells us how many rows, columns, and channels the image has.

3.2 Images as arrays, dimensions, channels, labels, and predictions

A digital image is an array of numbers. For a grayscale image, the shape is often (height, width). For a color image, the shape is usually (height, width, channels), where the channels are red, green, and blue, abbreviated as RGB. If a model receives a batch of images rather than one image, the full tensor shape becomes (batch, height, width, channels).

Term	Meaning	Mechanical Engineering interpretation
Height and width	Number of pixel rows and columns	How much spatial detail is visible in a crack, spall, or surface defect image
Channel	A separate layer of values, such as red, green, or blue	Color information can help separate rust stains, paint, water, or material texture
Label	The known class attached to an image during training	Cracked, intact, corrosion, surface defect, minor damage, major damage
Prediction	The class or region estimated by the model	The model's best guess for the engineering image
Probability	A confidence-like score assigned to a class	How strongly the model leans toward 'crack present' or another class name

A class name is the human-readable name associated with a numeric label. For example, label 0 might correspond to 'intact' and label 1 to 'cracked'. A probability is not the same thing as a hard decision. If a

model assigns probabilities [0.15, 0.85], it is leaning strongly toward the second class, but the engineering decision still depends on the chosen threshold and the cost of mistakes.

3.3 Training, validation, and testing for image data

The training split is used to fit model parameters. The validation split is used while developing the model, for example to monitor overfitting or choose settings such as learning rate and augmentation strength. The test split is kept separate for final evaluation. Keeping these roles separate reduces the risk of overestimating performance [10].

A simple split workflow is shown below. The keyword argument `stratify=labels` tries to keep the class proportions similar across splits.

```
from sklearn.model_selection import train_test_split

# image_paths and labels are simple Python lists in this toy example
train_paths, test_paths, y_train, y_test = train_test_split(
    image_paths, labels, test_size=0.20, stratify=labels, random_state=42
)

train_paths, val_paths, y_train, y_val = train_test_split(
    train_paths, y_train, test_size=0.20, stratify=y_train, random_state=42
)
```

In safety-critical Mechanical Engineering work, careless splitting can produce misleading results. Images from the same structure, date, or camera pass should not leak across splits if the goal is realistic generalization.

Mechanical Engineering interpretation. If photographs from the same mechanical component inspection day appear in both training and test data, the reported score may look stronger than true field performance. The model may have memorized repeated backgrounds, lighting, or surface textures rather than learning robust defect concepts.

3.4 Simple plotting and quick inspection

Before training any model, it is good practice to look at a few images, class labels, and counts. In engineering projects, many problems are visible immediately: mixed resolutions, wrong labels, blank images, or one class that is much rarer than the others.

```
import matplotlib.pyplot as plt

# images has shape (batch, height, width, channels)
for images, labels in train_ds.take(1):
    plt.figure(figsize=(8, 4))
    for i in range(4):
        plt.subplot(1, 4, i + 1)
        plt.imshow(images[i].numpy().astype("uint8"))
        plt.title(class_names[labels[i]])
        plt.axis("off")
    plt.tight_layout()
    plt.show()
```

A small visual check like this is often the fastest quality-control step in a beginner workflow.

Section summary. The minimum Python needed here is straightforward: arrays store images, labels store class names, probabilities express model confidence, and train-validation-test splits give structure to the learning process.

4 Deep learning mounting bases for image analysis

A convolutional neural network, abbreviated as CNN, is a neural network designed to process image-like arrays efficiently [2], [9]. Instead of connecting every pixel directly to every neuron in the first layer, a CNN uses small filters that slide across the image. Each filter looks for a local pattern such as an edge, a corner, a texture change, or a repeated surface motif. Stacking many such layers allows the network to build a hierarchy of features, from simple edges to more meaningful structures such as crack traces or corrosion regions.

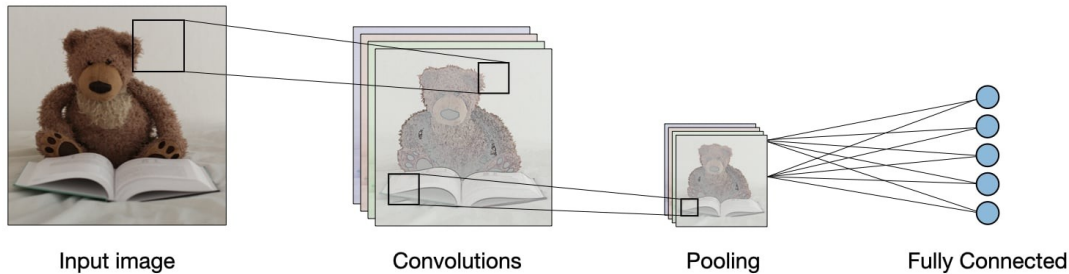


Figure 2. A high-level convolutional neural network pipeline: input image, convolutional feature extraction, pooling, and fully connected classification. Source: [2].

Figure 2 gives the big picture [2]. The front of the network extracts features from the image, and the final layers convert those features into a class prediction. For Mechanical Engineering, this means the network gradually transforms raw visual texture into a decision such as 'cracked' or 'intact'.

4.1 Filters, kernels, and feature maps

The words filter and kernel are often used almost interchangeably in beginner explanations. A small numeric kernel slides across the image and computes a weighted sum at each location. The resulting output is called a feature map. Each learned kernel specializes in a pattern that helps the training objective [2], [9].

$$y[i, j, k] = \sum_u \sum_v \sum_c x[i + u, j + v, c] w[u, v, c, k] + b_k$$

In this expression, x is the input image or feature tensor, w is the kernel weight tensor, b_k is the bias term for output channel k , and y is the output feature map. The important idea for beginners is that the same small kernel is reused across many spatial locations. That weight sharing makes convolution practical for large images.

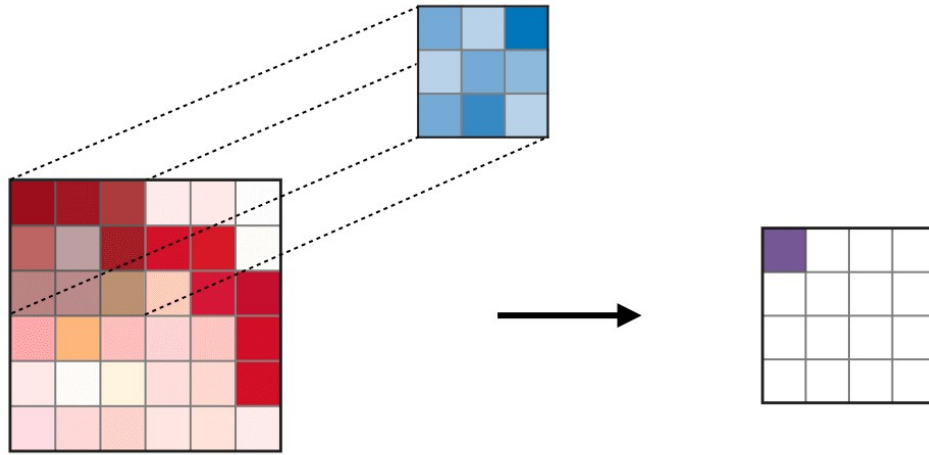


Figure 3. Example of a convolution operation. A local image patch is multiplied by learned kernel values to produce one output value in a feature map. Source: [2].

4.2 Activations, pooling, flattening, and dense classification heads

After a convolution computes a linear response, an activation function introduces nonlinearity. The most common beginner activation is the rectified linear unit, abbreviated as ReLU. It keeps positive values and sets negative values to zero. Pooling then reduces spatial size by summarizing small neighborhoods. Max pooling keeps the strongest response in each region. Average pooling keeps the mean value in each region [2], [9].

$$\text{ReLU}(z) = \max(0, z)$$

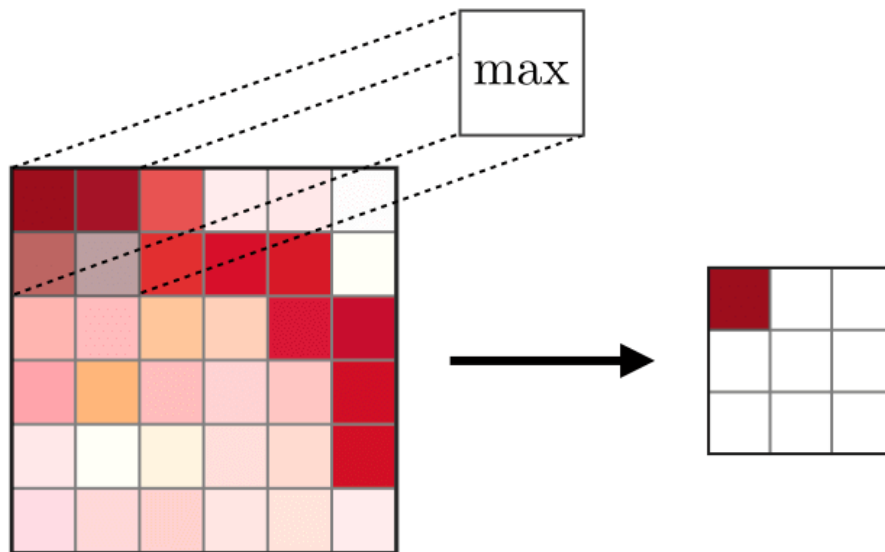


Figure 4. Max pooling keeps the strongest value in a local neighborhood, reducing image size while preserving salient responses. Source: [2].

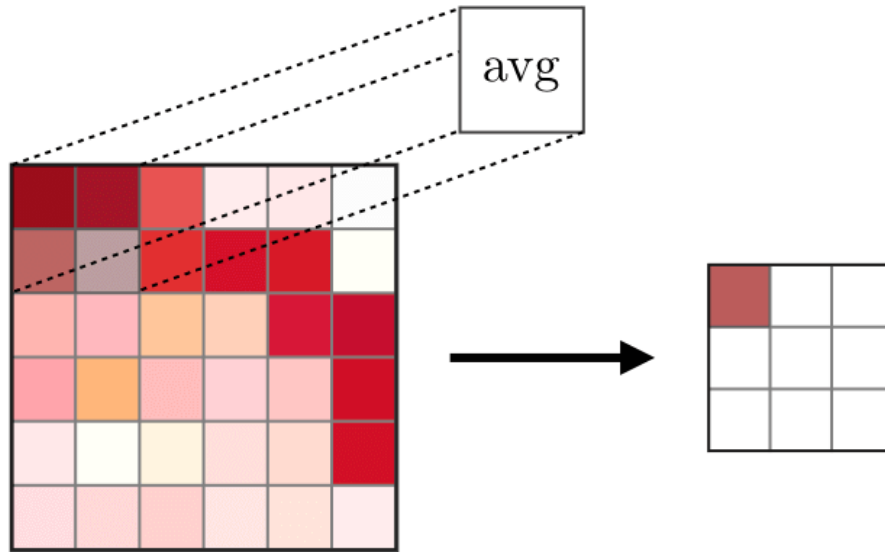


Figure 5. Average pooling replaces each neighborhood with its mean value. In transfer-learning heads, global average pooling often summarizes each feature channel over the whole image. Source: [2].

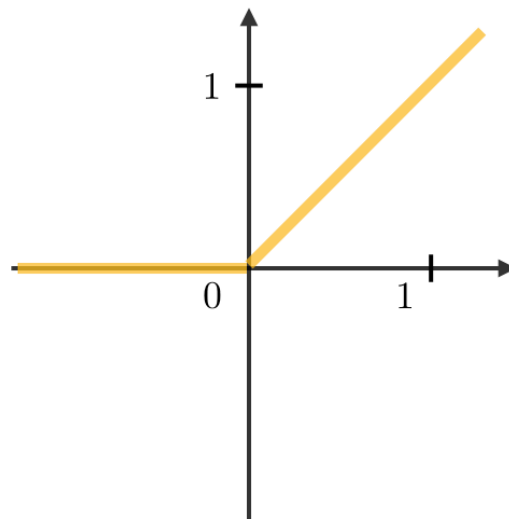


Figure 6. The rectified linear unit activation function, which outputs zero for negative inputs and leaves positive inputs unchanged. Source: [2].

Flattening or global pooling prepares the feature extractor output for the final classifier. A dense classification head then maps the extracted features into class scores. In a crack-versus-intact classifier, the head might output two class probabilities. In a three-level condition classifier, it might output probabilities for 'sound', 'moderate damage', and 'severe damage'.

4.3 Why convolution is useful for images

Convolution is useful because nearby pixels in an image are strongly related. A crack is not defined by one isolated pixel. It is defined by a spatial pattern that extends across neighboring pixels. A convolutional layer respects that local structure. It also learns translationally reusable filters, meaning that an edge detector learned in one image location can help at another location [2], [9].

- Local connectivity focuses the model on nearby pixel relationships.

- Weight sharing keeps the number of learnable parameters manageable.
- Stacked layers allow the model to move from simple features to more meaningful engineering patterns.
- Pooling or strided layers reduce image size so later layers can work with more abstract information.

4.4 Hierarchical feature learning

Early CNN layers often learn generic features such as edges, corners, and simple textures. Middle layers combine these into more complex motifs such as line segments, repeated surface grains, or contour fragments. Later layers assemble broader visual concepts such as a crack network, rust-stain texture, delamination region, or surface defect boundary [9], [10].

Mechanical Engineering interpretation. On a material surface, the model may first detect fine edge-like patterns, then connect them into elongated dark traces, and finally combine those traces with surrounding texture to decide whether the surface looks cracked. That is the beginner-friendly meaning of hierarchical feature learning.

4.5 Transfer learning and why it matters when engineering datasets are small

Transfer learning means starting from a model that has already learned useful visual features from a large source dataset and adapting it to a new task [3], [10]. This is extremely valuable in Mechanical Engineering because many inspection datasets are modest in size. A student may have only a few thousand labeled images or less. Starting from a pre-trained network often leads to better accuracy, faster convergence, and more stable training than starting from random weights.

- Freeze the pre-trained feature extractor first, then train only a small classification head.
- After the head becomes stable, optionally fine-tune the upper layers with a very small learning rate.
- Use augmentation to simulate modest variation in view angle, scale, lighting, or orientation.
- Monitor validation results carefully so that fine-tuning improves generalization rather than overfitting.

Section summary. A convolutional neural network learns visual patterns through filters, activations, pooling, and a final decision layer. Transfer learning is especially important in Mechanical Engineering because labeled defect datasets are often limited.

5 Lecture Part I: Image datasets and convolutional neural networks

Part I focuses on the complete beginner workflow for image classification in Mechanical Engineering. The central question is simple: given an inspection image, can the model predict the correct class reliably enough to support engineering review? To answer that question well, students must understand image data preparation, transfer learning, and safety-aware evaluation.

5.1 Image data preparation

Image learning begins with organized data. In a beginner project, the dataset should be easy to inspect, easy to split, and easy to explain to another engineer. Good organization reduces mistakes later in training.

5.1.1 Folder structures and class-labeled image organization

A common beginner-friendly approach is to group images by class name. For binary crack classification, one folder may contain cracked images and another may contain intact images. For a multi-class asset-condition task, folders may represent classes such as sound, minor_damage, and severe_damage.

A clean folder layout keeps the class meaning visible at a glance.

```
inspection_dataset/  
  train/  
    cracked/  
    intact/  
  val/  
    cracked/  
    intact/  
  test/  
    cracked/  
    intact/
```

A second common approach is to keep one top-level dataset folder with one subfolder per class and let the library create the training-validation split automatically.

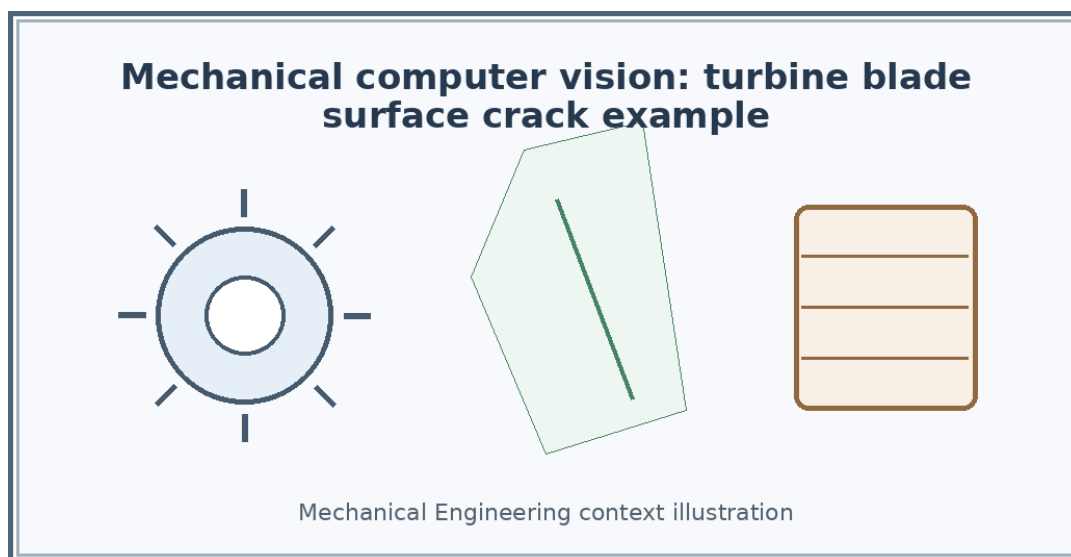


Figure 7. Example surface patches from a mechanical-mechanical systems crack dataset: a cracked housing image and an uncracked housing image. Source: [8].

Figure 6 shows why class labeling must be visually consistent [8]. If 'cracked' images contain hairline cracks, wide cracks, shadows, and paint marks, while 'uncracked' images are captured under completely different conditions, the model may learn camera or lighting shortcuts rather than true damage patterns.

5.1.2 Training and validation splits, image loading, resizing, and normalization

A model usually expects a fixed image size, for example 224 by 224 pixels. Resizing brings images to a common size. Normalization rescales pixel values so training is more numerically stable. A common beginner choice is to divide raw pixel values by 255 so that the data fall in the interval from 0 to 1.

The TensorFlow utility below loads a directory-organized dataset and returns batches of images and labels.

```
import tensorflow as tf

image_size = (224, 224)
batch_size = 32

train_ds = tf.keras.utils.image_dataset_from_directory(
    "inspection_dataset",
    validation_split=0.20,
    subset="training",
    seed=42,
    image_size=image_size,
    batch_size=batch_size
)

val_ds = tf.keras.utils.image_dataset_from_directory(
    "inspection_dataset",
    validation_split=0.20,
    subset="validation",
    seed=42,
    image_size=image_size,
    batch_size=batch_size
)

class_names = train_ds.class_names
print("Classes:", class_names)

for images, labels in train_ds.take(1):
    print("Image batch shape:", images.shape)
    print("Label batch shape:", labels.shape)
```

The image batch shape is usually (batch, height, width, channels). If the batch size is 32, then a typical output shape is (32, 224, 224, 3).

5.1.3 Data augmentation basics

Data augmentation creates modified versions of training images so the model sees more variation [3], [4]. In Mechanical Engineering this is especially helpful when the dataset is small. Mild rotations, zoom changes, flips, and contrast adjustments can make the model less brittle to small changes in camera pose or illumination. However, augmentation should remain physically sensible. For example, a vertical flip may be acceptable for some surface textures but less meaningful for scenes where gravity-dependent context matters.

Augmentation	What it changes	Mechanical Engineering benefit	Caution
Flip	Reflects the image horizontally or vertically	Improves invariance to camera direction on texture-like surfaces	Avoid if orientation itself carries meaning
Rotation	Turns the image by a small angle	Helps with varied camera pose during field capture	Large rotations can create unrealistic viewpoints
Zoom	Mildly crops in or out	Helps when defect size varies	Strong zoom may cut away needed context
Contrast adjustment	Changes brightness or	Helps under varying lighting	Excessive change can

	contrast relationships	conditions	distort material appearance
--	------------------------	------------	-----------------------------

TensorFlow provides augmentation layers that can be inserted directly into the model or applied to a batch during inspection.

```
data_augmentation = tf.keras.Sequential([
    tf.keras.layers.RandomFlip("horizontal"),
    tf.keras.layers.RandomRotation(0.05),
    tf.keras.layers.RandomZoom(0.10),
    tf.keras.layers.RandomContrast(0.10),
])

for images, labels in train_ds.take(1):
    augmented = data_augmentation(images, training=True)
    print("Augmented batch shape:", augmented.shape)
```

The important beginner idea is that augmentation should be applied to the training data only, not to the validation or test data.

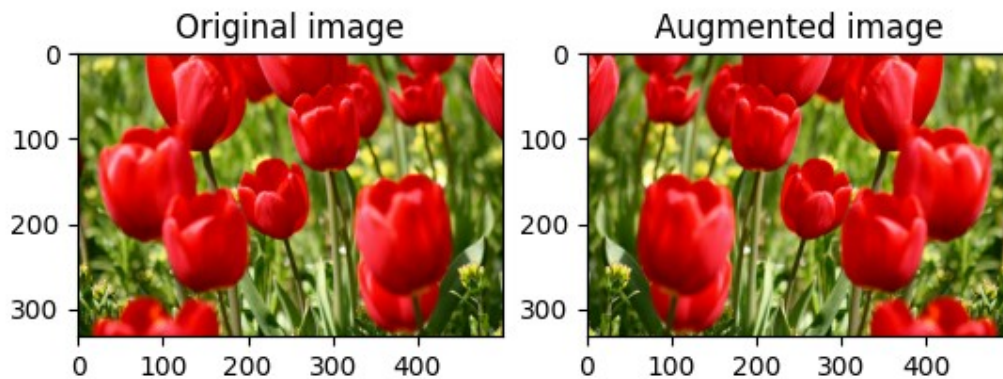


Figure 8. An official TensorFlow augmentation example showing how a flip operation changes the appearance of a training image while preserving the class meaning. Source: [4].

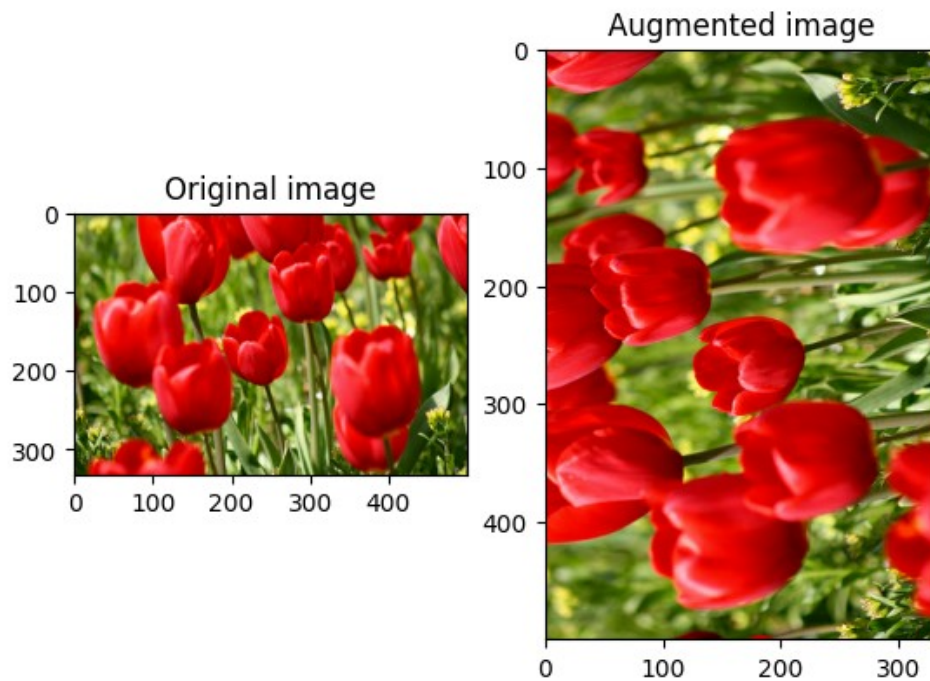


Figure 9. An official TensorFlow augmentation example showing small rotations that help a model tolerate modest viewpoint variation. Source: [4].

5.1.4 Visualizing sample images and summarizing class counts

A beginner should never train blindly. Before model fitting, inspect representative images from each class and count how many examples belong to each label. This step can reveal imbalance, mislabeled images, or split mistakes early.

```
from collections import Counter
import matplotlib.pyplot as plt

label_counter = Counter()

for _, labels in train_ds:
    label_counter.update(labels.numpy().tolist())

print("Training class counts:", label_counter)

for images, labels in train_ds.take(1):
    plt.figure(figsize=(8, 4))
    for i in range(6):
        plt.subplot(2, 3, i + 1)
        plt.imshow(images[i].numpy().astype("uint8"))
        plt.title(class_names[labels[i]])
        plt.axis("off")
    plt.tight_layout()
    plt.show()
```

A class count summary is especially important in defect inspection because rare but critical classes often receive fewer images.

Mechanical Engineering interpretation. Suppose 95 percent of the training images are intact surfaces and only 5 percent contain cracking. A model can appear accurate while still missing many real defects. The class counts should be reviewed before the first epoch of training.

5.2 Transfer learning

Transfer learning is often the most practical starting point for Mechanical Engineering image classification [3]. The strategy is to reuse a CNN that has already learned general-purpose visual features, then adapt it to the engineering task.

5.2.1 Comparing a lightweight custom baseline with a pre-trained model

It is educational to begin with a small custom CNN because it makes the mechanics of convolution, pooling, and dense classification visible. However, when the dataset is limited, a pre-trained model usually performs better because it starts from much richer visual features.

A custom baseline can be kept intentionally small.

```
from tensorflow import keras
from tensorflow.keras import layers

num_classes = len(class_names)

baseline_model = keras.Sequential([
    layers.Rescaling(1./255, input_shape=(224, 224, 3)),
    layers.Conv2D(32, 3, activation="relu"),
    layers.MaxPooling2D(),
    layers.Conv2D(64, 3, activation="relu"),
    layers.MaxPooling2D(),
    layers.Conv2D(128, 3, activation="relu"),
    layers.GlobalAveragePooling2D(),
    layers.Dense(64, activation="relu"),
    layers.Dense(num_classes, activation="softmax")
])

baseline_model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
baseline_model.summary()
```

This baseline is useful for learning, but it usually has fewer visual priors than a pre-trained model such as MobileNetV2 or EfficientNet.

Approach	Strength	Limitation	Good beginner use
Small custom CNN	Transparent and easy to understand	Needs more data and more tuning	Learning the workflow and creating a reference baseline
Pre-trained CNN	Usually stronger performance on limited data	Requires transfer-learning concepts	Mechanical defect datasets with modest size

5.2.2 Freezing base layers and training only the classifier head

In the first transfer-learning stage, the pre-trained base model is frozen. This means its weights are not updated. Only the new classification head learns. This is called feature extraction [3].

```
from tensorflow import keras
from tensorflow.keras import layers

base_model = keras.applications.MobileNetV2(
    input_shape=(224, 224, 3),
    include_top=False,
    weights="imagenet"
```

```

)
base_model.trainable = False

inputs = keras.Input(shape=(224, 224, 3))
x = data_augmentation(inputs)
x = keras.applications.mobilenet_v2.preprocess_input(x)
x = base_model(x, training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dropout(0.20)(x)
outputs = layers.Dense(num_classes, activation="softmax")(x)

model = keras.Model(inputs, outputs)

model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
model.summary()

```

The input is a batch of images with shape (batch, 224, 224, 3). The output is a batch of class probabilities with shape (batch, num_classes).

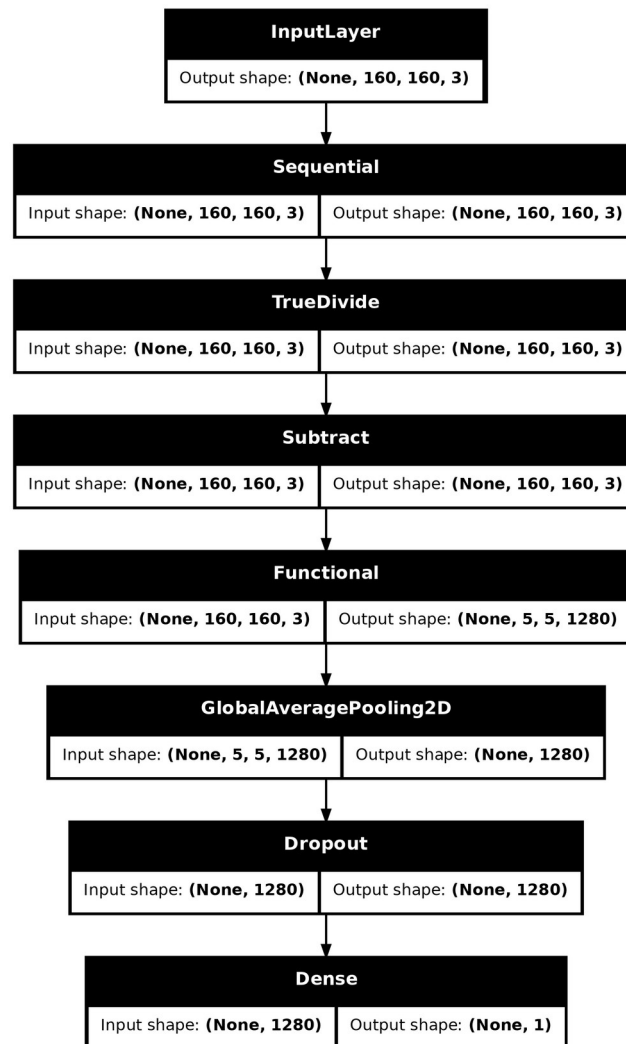


Figure 10. Official TensorFlow model diagram for a transfer-learning classifier head built on a pre-trained feature extractor, followed by global average pooling, dropout, and a dense output layer. Source: [3].

5.2.3 Fine-tuning selected upper layers

After the head has learned stable class separation, some of the upper layers of the base model can be unfrozen and fine-tuned with a much smaller learning rate [3]. This allows the model to adapt more specifically to Mechanical Engineering textures and defect patterns.

```
base_model.trainable = True

# Freeze most of the base model, fine-tune only the upper layers
for layer in base_model.layers[:-30]:
    layer.trainable = False

model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-5),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

history_finetune = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=10
)
```

Fine-tuning should be done carefully. If the learning rate is too large, the useful pre-trained features can be damaged.

5.2.4 Why transfer learning is especially useful for crack and defect classification

Mechanical Engineering datasets are often small because labeling requires expert time. Crack images can vary by material texture, lighting, camera angle, paint, moisture, and background clutter. A pre-trained network already knows many generic image primitives such as edges, junctions, contours, and textures. Transfer learning therefore offers a strong starting point for engineering image tasks [3], [10].

Practical advice. Train a small baseline first so you understand the pipeline, then move to transfer learning for the strongest realistic beginner result. Compare frozen and fine-tuned performance with the same validation split and the same evaluation metrics.

5.3 Classifier evaluation in safety-critical Mechanical Engineering contexts

A classifier is not judged only by overall accuracy. In engineering inspection, the meaning of each mistake matters. A false negative, where real damage is missed, may be much more serious than a false positive, where an intact image is flagged for extra review.

5.3.1 Confusion matrices

A confusion matrix counts how many examples from each true class were predicted as each output class [6]. It is one of the most useful beginner tools because it shows where the model fails.

```
import numpy as np
from sklearn.metrics import confusion_matrix, classification_report

y_true = []
y_prob = []

for images, labels in val_ds:
    probs = model.predict(images, verbose=0)
    y_true.extend(labels.numpy())
    y_prob.extend(probs)

y_true = np.array(y_true)
y_prob = np.array(y_prob)
```

```

y_pred = np.argmax(y_prob, axis=1)

cm = confusion_matrix(y_true, y_pred)
print(cm)
print(classification_report(y_true, y_pred, target_names=class_names))

```

The confusion matrix output lets you inspect which classes the model confuses. In a crack classifier, the key concern is how often cracked images are predicted as intact.

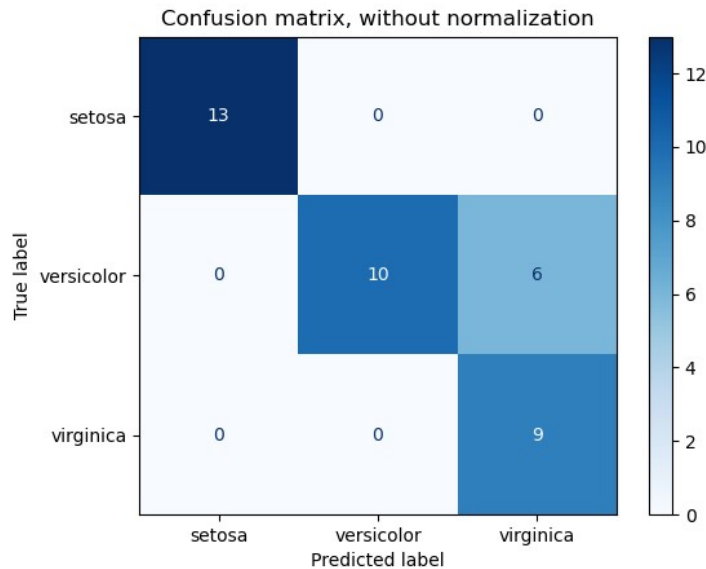


Figure 11. Example confusion matrix from the scikit-learn documentation. The diagonal cells represent correct predictions, while off-diagonal cells represent mistakes. Source: [6].

5.3.2 Accuracy, precision, recall, and F1-score

Accuracy is the fraction of all predictions that are correct. Precision asks: when the model predicts a defect, how often is it right? Recall asks: among all real defect images, how many did the model successfully find? The F1-score balances precision and recall [6], [7]. In safety-sensitive Mechanical Engineering applications, recall is often especially important because missed damage can be costly.

$$Accuracy = (TP + TN) / (TP + TN + FP + FN)$$

$$Precision = TP / (TP + FP)$$

$$Recall = TP / (TP + FN)$$

$$F1 = 2 \cdot Precision \cdot Recall / (Precision + Recall)$$

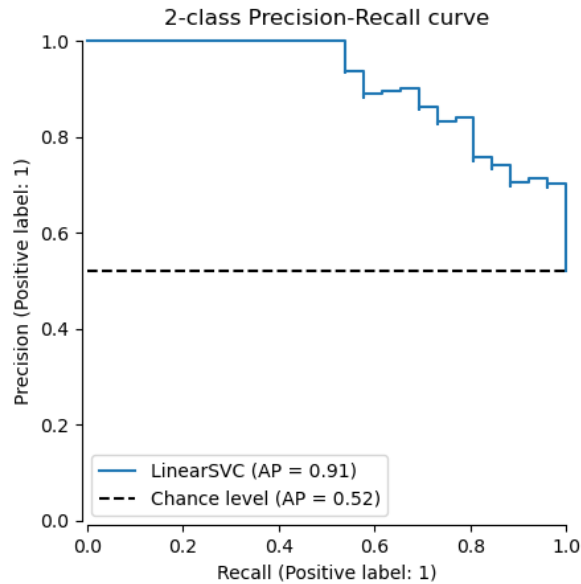


Figure 12. Example precision-recall curve. The curve helps students see how precision and recall trade off as the decision threshold changes. Source: [7].

Metric	What it emphasizes	Why it matters in engineering
Accuracy	Overall correctness	Useful, but can hide poor defect detection when classes are imbalanced
Precision	Low false-alarm rate among predicted defects	Important when follow-up inspection is costly
Recall	Low miss rate for true defects	Often critical when missed damage creates safety risk
F1-score	Balance of precision and recall	Useful when both kinds of error matter

5.3.3 False negatives, class imbalance, and balancing remedies

In many inspection datasets, serious defects are rarer than intact surfaces. This creates class imbalance. A model can achieve high accuracy by over-predicting the majority class. That behavior is dangerous when the minority class is the one that most requires action.

One simple balancing remedy is to give the rarer class a larger training weight.

```
# Example class weights for a binary crack classifier
# Class 0: intact, Class 1: cracked
class_weight = {
    0: 1.0,
    1: 4.0
}

history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=10,
    class_weight=class_weight
)
```

Other practical remedies include collecting more rare-defect images, targeted augmentation, or careful resampling. The correct choice depends on the dataset and the engineering objective.

Class weighting tells the training process that mistakes on the minority class should count more strongly. In a defect-screening application, this can increase recall for the defect class, though sometimes at the cost of more false alarms.

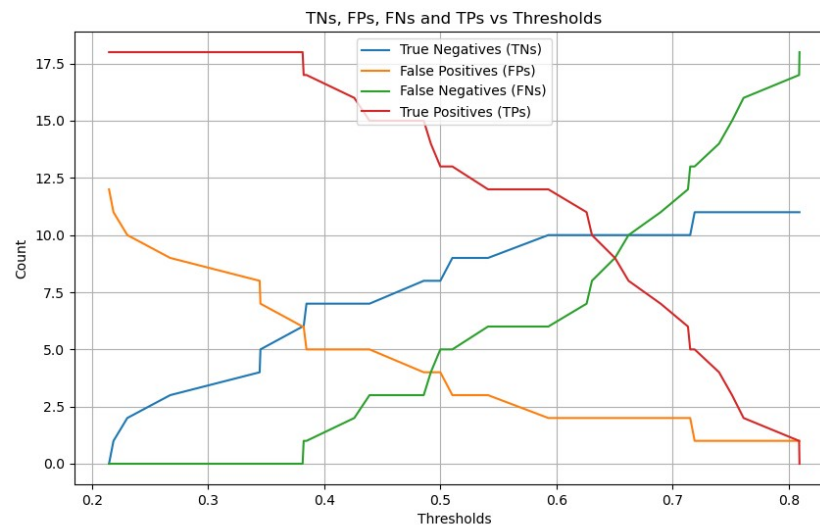


Figure 13. Example threshold-versus-metric plot. As the confidence threshold changes, true positives, false positives, true negatives, and false negatives change as well. Source: [6].

A threshold plot is valuable because the default threshold is not always the best engineering choice [6]. If the cost of missing a defect is high, a lower threshold may be preferred so that more uncertain cases are sent for human review.

5.3.4 Before-and-after comparisons when balancing is applied

A strong beginner habit is to compare the confusion matrix and recall before and after applying class weighting or another imbalance remedy. The central question is whether the model becomes safer for the engineering task. A small decrease in overall accuracy may be acceptable if recall for true defects rises substantially.

Mechanical Engineering interpretation. If a machined surface-defect classifier improves crack recall from 0.58 to 0.84 after class weighting, that may be a worthwhile engineering trade even if precision falls slightly. The key is to report the trade honestly.

5.4 Theory and mathematics for image classification

The mathematics needed at beginner level is modest. The purpose of the formulas below is not to make the tutorial abstract. The purpose is to make the code less mysterious.

5.4.1 Convolution, feature maps, and local pattern learning

Convolution can be read as a repeated local weighted sum. A small kernel slides over the image and measures how strongly a local pattern is present [2], [9]. When the kernel resembles an edge detector, high responses appear where the image contains similar edge structure.

$$\text{Feature map value} = \text{local patch} \cdot \text{kernel} + \text{bias}$$

For Mechanical Engineering images, such local patterns may correspond to thin crack edges, rust boundaries, spall texture changes, or surface defect borders. The same kernel is reused across the image, which is why a crack-like edge can be recognized at many positions.

5.4.2 ReLU, softmax probabilities, and cross-entropy loss

ReLU introduces nonlinearity so the network can represent more than a simple linear filter. Softmax converts the final score vector into class probabilities. Cross-entropy loss measures how well those probabilities align with the true labels [9], [10].

$$p_i = \exp(z_i) / \sum_j \exp(z_j)$$

$$\text{Cross-entropy: } L = - \sum_i y_i \log(p_i)$$

Here z_i is the class score for class i , p_i is the predicted probability, and y_i is the true-label indicator. In plain language, cross-entropy penalizes the model when it assigns low probability to the correct class.

5.4.3 Confusion matrix quantities, precision, recall, and class imbalance effects

In a binary defect problem, the four counts are true positives, true negatives, false positives, and false negatives. Class imbalance matters because the majority class can dominate overall accuracy. That is why class-sensitive metrics must be reported alongside accuracy [6], [7].

Summary of Part I. A reliable image-classification workflow in Mechanical Engineering begins with organized image data, appropriate augmentation, and careful transfer learning. Performance must be interpreted with confusion matrices and class-sensitive metrics, especially when missed defects are costly.

6 Lecture Part II: Detection and visualization

Part II broadens the scope from whole-image classification to spatial reasoning. In classification, the model answers a question about the entire image. In detection and segmentation, the model must show where the relevant object or defect is located. This difference is crucial in Mechanical Engineering because inspectors often need visible evidence, not only a label.

6.1 Computer vision task types

The four beginner task types below answer different questions [5]. Choosing the wrong task definition is a common source of confusion.

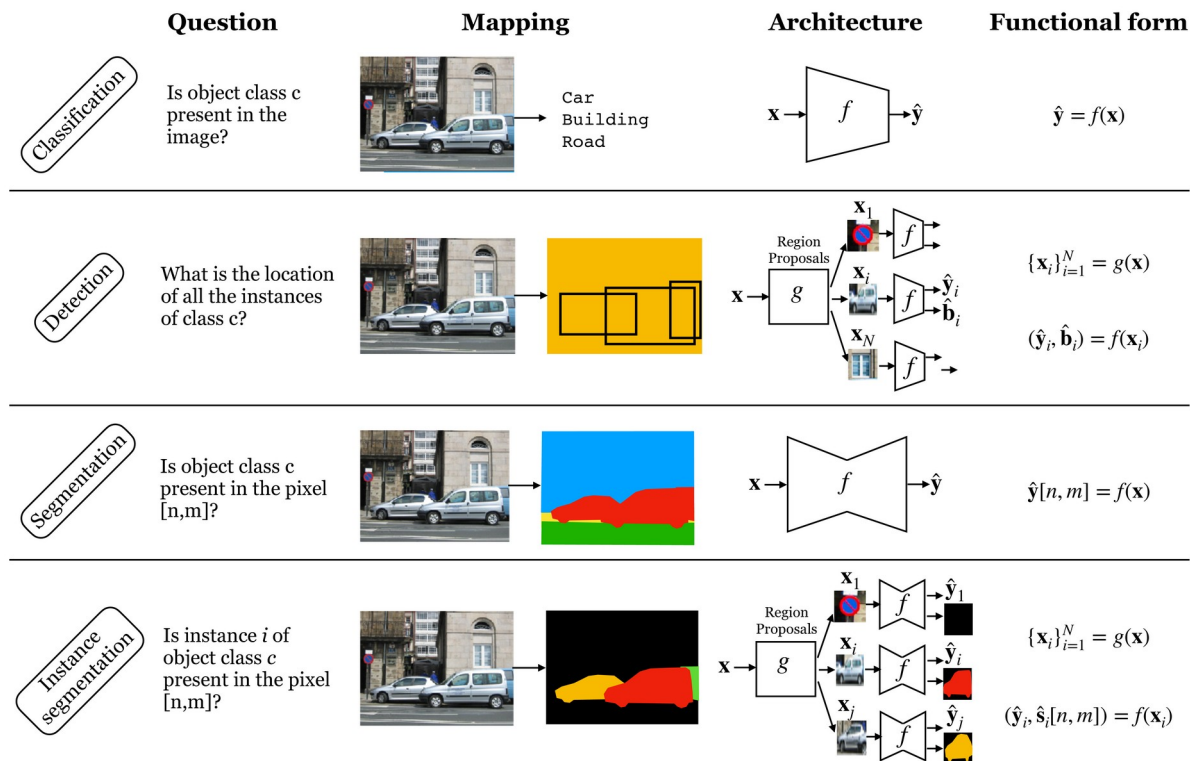


Figure 14. A comparison of four vision task types: classification, detection, segmentation, and instance segmentation.
Source: [5].

Task type	Input	Output	When it is useful in Mechanical Engineering
Image classification	One image	One class label or class-probability vector	Cracked versus intact surface screening, condition class prediction
Object detection	One image	Boxes, class names, and confidence scores	Surface defect localization, damaged-joint localization, object-level inventory
Semantic segmentation	One image	A class label for each pixel	Corrosion region mapping, water-stain region extraction, damage-area measurement
Instance segmentation	One image	Separate mask for each object instance	Distinguishing multiple surface defects or multiple spall regions in the same image

The engineering question should drive the task choice. If the question is simply 'Does this image contain a defect?', classification is enough. If the inspector needs to know where the defect is, detection or segmentation is needed. If separate objects must be counted or measured individually, instance segmentation becomes attractive [5].

6.1.1 Image classification versus localization and detection

Classification reports whether a class is present in the image. Localization adds a rough spatial answer by drawing a box around the relevant object. Object detection extends that idea to multiple objects and classes [2], [5].

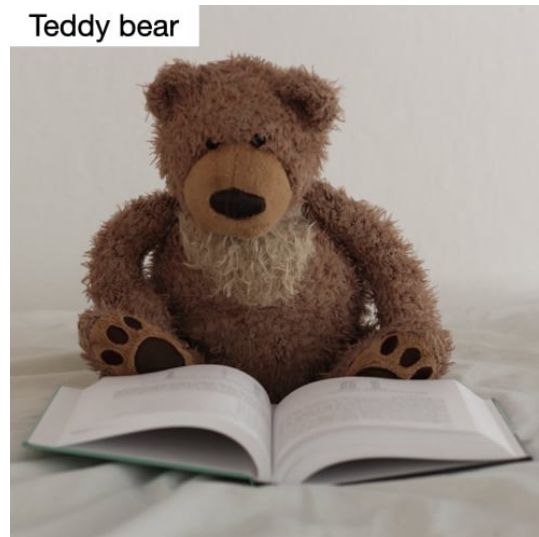


Figure 15. Example of classification and rough localization on an image. The output moves from 'what is present?' toward 'where is it?' Source: [2].

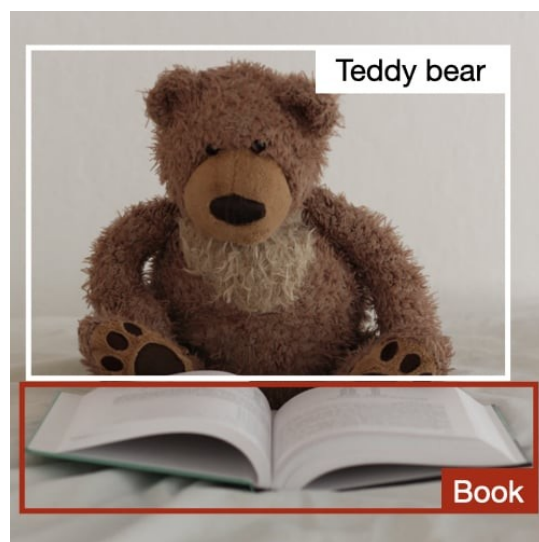


Figure 16. Example of object detection. Multiple boxes can be predicted, each with its own class and confidence. Source: [2].

6.2 Simple detection and segmentation workflows

A beginner does not need to train a detector from scratch to understand the workflow. It is enough to know what goes in, what comes out, and how those outputs are filtered and displayed.

6.2.1 Pre-trained detection models and running inference

A pre-trained detector receives an image and returns candidate boxes, class IDs, and confidence scores. In real Mechanical Engineering practice, the classes might be surface defect, rust patch, spall, exposed steel, or loose component. The code below illustrates a beginner-friendly inference pattern using a common high-level detector interface.

```
from ultralytics import YOLO

# A small pre-trained detector
detector = YOLO("yolov8n.pt")

results = detector.predict(
    source="component_inspection.jpg",
    conf=0.25,
    imgsz=640
)

result = results[0]
boxes = result.boxes.xyxy.cpu().numpy()      # [x1, y1, x2, y2]
scores = result.boxes.conf.cpu().numpy()
class_ids = result.boxes.cls.cpu().numpy().astype(int)

print("Number of detections:", len(boxes))
```

The inputs are one or more images. The outputs are boxes, confidence scores, and predicted class IDs. Different packages expose these outputs differently, but the logical structure is similar.

6.2.2 Filtering low-confidence boxes and non-maximum suppression

Detectors often produce many overlapping boxes. Low-confidence boxes may be filtered out first. Then a rule called non-maximum suppression, abbreviated as NMS, removes redundant overlapping boxes so that the final output is cleaner [2].

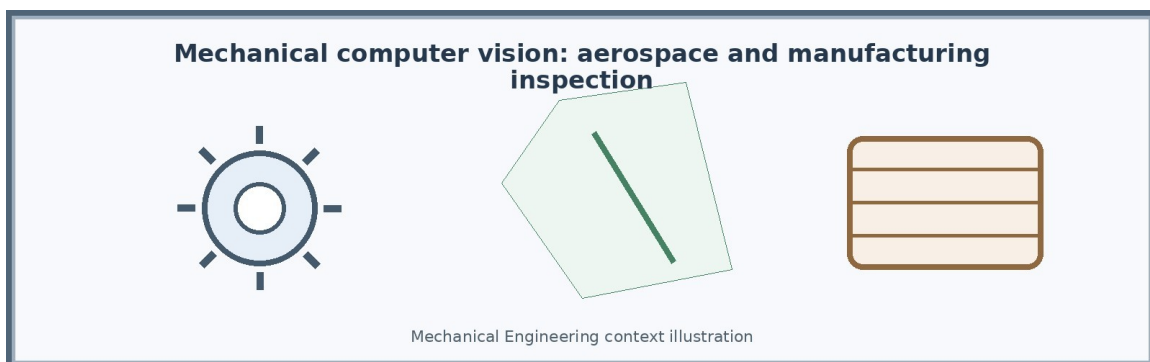


Figure 17. Illustration of non-maximum suppression. Overlapping candidate boxes are reduced to a smaller final set of detections. Source: [2].

Even after a detector has run, the engineer still chooses a reporting threshold.

```
# Keep only detections above a higher reporting threshold
keep = scores >= 0.40
filtered_boxes = boxes[keep]
filtered_scores = scores[keep]
filtered_class_ids = class_ids[keep]
```

A lower threshold catches more possible defects but may produce more false alarms. A higher threshold produces fewer boxes but may miss subtle damage.

6.2.3 Segmentation mask generation and per-pixel reasoning

A semantic segmentation model outputs a class prediction for each pixel. Instead of drawing one box, it returns a mask. This is helpful when the engineering question depends on the area, shape, or extent of deterioration rather than only its approximate location [5].

```
# Example segmentation output shape:  
# (batch, height, width, num_classes)  
  
mask_prob = segmentation_model.predict(image_batch)[0]  
pred_mask = mask_prob.argmax(axis=-1)  
  
print("Predicted mask shape:", pred_mask.shape)
```

If `pred_mask` has shape (256, 256), then each element is a pixel-level class ID. For example, class 0 may mean background, class 1 may mean corrosion, and class 2 may mean spalling.

Semantic segmentation groups all pixels of the same class together. Instance segmentation goes one step further and separates individual objects into distinct masks [5]. This matters when multiple surface defects or multiple damage regions must be counted individually.

6.2.4 One-stage versus two-stage detection intuition

A one-stage detector predicts boxes and classes in a single pass across the image. A two-stage detector first proposes candidate regions, then classifies and refines them [2]. For beginners, the main point is not the algorithmic detail. The main point is the tradeoff: one-stage methods are often faster, while two-stage methods may provide stronger localization accuracy in some settings.

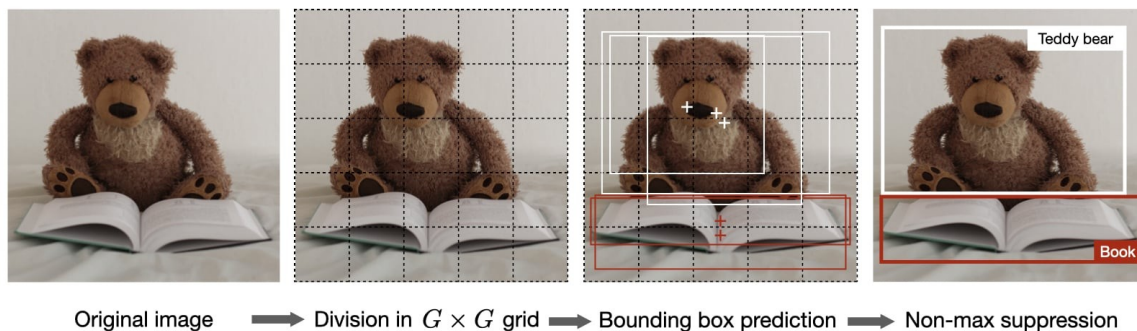


Figure 18. Illustration of a one-stage detector workflow using the YOLO idea: the image is divided into regions and boxes are predicted directly. Source: [2].

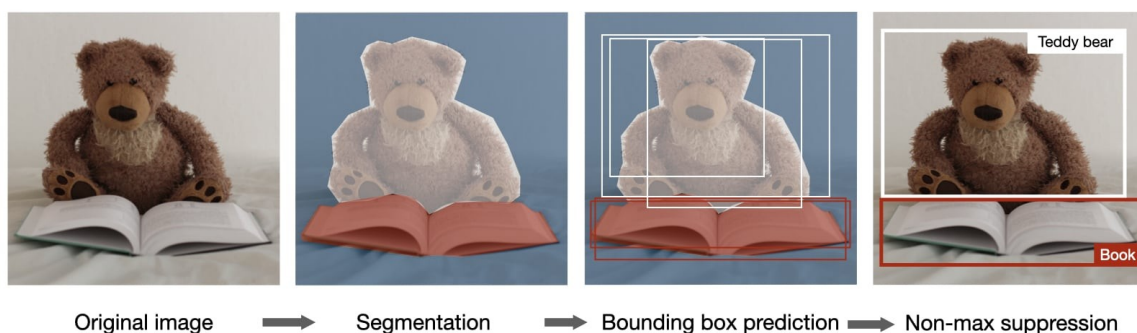


Figure 19. Illustration of a two-stage R-CNN style detector workflow: candidate regions are proposed first and then classified and refined. Source: [2].

6.3 Evaluation ideas for detection and segmentation

Detection and segmentation need more than plain classification metrics. A detector must not only say the correct class, it must also place the predicted region in the correct location. That is why matching logic and overlap measures are required.

6.3.1 Prediction-to-ground-truth matching and Intersection over Union

A detection is usually counted as correct only if the class is correct and the predicted box overlaps the true box strongly enough. The most common overlap measure is Intersection over Union, abbreviated as IoU [2].

$$IoU = \text{area}(\text{predicted box} \cap \text{true box}) / \text{area}(\text{predicted box} \cup \text{true box})$$

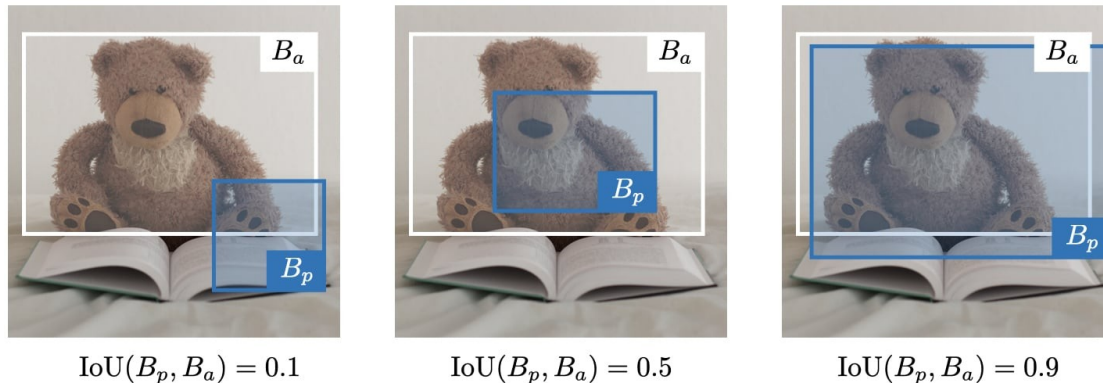


Figure 20. Intersection over Union compares the overlap between a predicted box and the true box. Larger overlap means a better localization match. Source: [2].

If the IoU is below the chosen threshold, the prediction may be treated as a miss even if the class name is correct. This is why detection evaluation is stricter than whole-image classification.

6.3.2 Precision, recall, and confidence thresholds for detections

After matching predicted boxes to true boxes, the same ideas of precision and recall can be reused at the detection level. Precision asks how many reported boxes are correct. Recall asks how many true objects were found. Confidence thresholds shift the balance between these two quantities [7].

- Low threshold: more boxes reported, usually higher recall, but often lower precision.
- High threshold: fewer boxes reported, often higher precision, but sometimes lower recall.
- For safety-critical inspection, a lower threshold may be acceptable if uncertain boxes are reviewed by a human inspector.

6.3.3 Segmentation mask comparison

For segmentation, overlap can also be measured on masks rather than boxes. A simple beginner metric is the Jaccard score, which is mathematically the same idea as IoU but applied to the predicted and true pixel sets for a class.

```
import numpy as np

def mask_iou(pred_mask, true_mask, class_id):
    pred = pred_mask == class_id
    truth = true_mask == class_id

    intersection = np.logical_and(pred, truth).sum()
    union = np.logical_or(pred, truth).sum()

    return intersection / max(union, 1)

score = mask_iou(pred_mask, true_mask, class_id=1)
```

```
print("IoU for corrosion class:", score)
```

The score ranges from 0 to 1. A value near 1 means the predicted mask overlaps the true region very well.

6.4 Inspector-friendly visual outputs

A useful Mechanical Engineering model must produce outputs that humans can read quickly and trust. For inspection work, this often means overlaying predictions on the original image, using consistent colors, showing confidence scores, and placing original and annotated views side by side.

6.4.1 Drawing boxes, masks, labels, legends, and confidence scores

The drawing step is where technical output becomes engineering communication. A box without a class label or confidence score is incomplete. A mask without a legend is hard to interpret.

```
from PIL import Image, ImageDraw

image = Image.open("component_inspection.jpg").convert("RGB")
draw = ImageDraw.Draw(image)

class_names = {0: "surface defect", 1: "spall", 2: "corrosion"}

for box, score, cls_id in zip(filtered_boxes, filtered_scores, filtered_class_ids):
    x1, y1, x2, y2 = box.astype(int)
    label = class_names[int(cls_id)]
    text = f"{label}: {score:.2f}"

    draw.rectangle([x1, y1, x2, y2], outline="red", width=3)
    draw.text((x1, max(y1 - 15, 0)), text, fill="red")

image.show()
```

The exact colors and fonts can be standardized across a project so that all inspectors see the same visual conventions.

6.4.2 Overlaying segmentation masks and producing side-by-side panels

For segmentation, a transparent overlay helps inspectors compare the predicted region with the original surface appearance. Side-by-side panels are especially effective in reports because the evidence and the interpretation remain visually linked.

```
import matplotlib.pyplot as plt
import numpy as np

overlay = original_image.copy()
overlay[pred_mask == 1] = [255, 0, 0] # red for corrosion
overlay[pred_mask == 2] = [255, 255, 0] # yellow for spalling

fig, axes = plt.subplots(1, 3, figsize=(12, 4))

axes[0].imshow(original_image)
axes[0].set_title("Original")
axes[0].axis("off")

axes[1].imshow(overlay, alpha=0.75)
axes[1].set_title("Annotated overlay")
axes[1].axis("off")

axes[2].imshow(pred_mask, cmap="viridis")
axes[2].set_title("Predicted mask")
axes[2].axis("off")
```

```
plt.tight_layout()
plt.show()
```

The original panel preserves raw evidence, the overlay panel aids interpretation, and the mask panel gives a direct model output view.

6.4.3 Packaging annotated images for reports and inspection workflows

In professional use, annotated images should follow consistent rules. The same defect type should always use the same color. Confidence scores should be reported in a consistent format. Original and annotated images should appear together. Any uncertainty or need for human confirmation should be stated clearly.

Visual element	Recommended practice	Reason
Bounding boxes	Use consistent color by defect type	Helps rapid visual scanning across many pages
Masks	Use transparent overlays with a legend	Preserves the original surface texture underneath
Confidence scores	Display near the object or in a legend	Makes model certainty visible rather than hidden
Original versus annotated view	Show both side by side when possible	Supports traceable inspection communication
Class names	Use short, standardized terms	Reduces ambiguity in maintenance records

Mechanical Engineering interpretation. An inspector-ready output should answer three questions quickly: what was found, where was it found, and how confident is the model? If any of those are missing, the output is less useful in practice.

Summary of Part II. Detection and segmentation extend image classification by adding location or region information. Their evaluation depends on overlap measures such as IoU, and their real engineering value depends heavily on clear, consistent visual presentation.

7 Mechanical Engineering mini-cases and a practical workflow checklist

The best way to consolidate beginner knowledge is to revisit the main ideas through short Mechanical Engineering cases. Each case below emphasizes the link between the model output and the engineering question.

7.1 Cracked versus intact surface classification

Input: one surface image. Output: a class label such as cracked or intact, plus class probabilities. Best use: rapid screening when the goal is to triage images for further human review. Main risk: false negatives if the cracked class is rare or visually subtle. Good metrics: confusion matrix, precision, recall, and threshold-sensitive analysis.

7.2 Surface defect localization on machined surface images

Input: one production line image. Output: one or more bounding boxes with surface defect confidence scores. Best use: asset screening and maintenance prioritization when approximate location matters. Main risk: small surface defects may be missed if the threshold is too high or the image resolution is too low. Good metrics: detection precision, detection recall, and IoU-aware matching.

7.3 Corrosion-region or spall-region highlighting

Input: one structural image. Output: a pixel-level mask that highlights corrosion, spalling, or another defect class. Best use: area measurement, damage visualization, or documentation of deterioration extent. Main risk: region boundaries may be noisy or inconsistent across lighting conditions. Good metrics: mask IoU and class-wise region comparison.

7.4 Mechanical component or wind tunnel test-section inspection imagery for annotated reporting

Input: an inspection photograph captured in the field. Output: original image, annotated image, confidence score, and short notes. Best use: communication and traceability. Main risk: visually inconsistent annotations can confuse human reviewers. Good practice: preserve the original evidence, use a standardized legend, and record when the model output requires human confirmation.

7.5 A practical beginner workflow checklist

9. Define the engineering question clearly: classification, detection, segmentation, or instance segmentation.
10. Inspect the images manually before training. Look for mixed resolutions, mislabeled classes, poor lighting, repeated scenes, and class imbalance.
11. Create clean train-validation-test splits that reflect the intended deployment setting.
12. Start with a small baseline so the workflow is understandable.
13. Use transfer learning when the dataset is limited.
14. Track class-sensitive metrics, not only accuracy.
15. Tune the reporting threshold with the engineering consequence of false negatives and false positives in mind.
16. Produce visual outputs that preserve the original image and make the model's reasoning inspectable.
17. Document limitations honestly and keep a human review step for safety-critical decisions.

One-sentence takeaway. In Mechanical Engineering, the strongest beginner computer-vision workflow is not the one with the most complicated model. It is the one that produces stable training, interpretable outputs, and responsible engineering communication.

8 Conclusion

Computer vision gives Mechanical Engineers a practical way to turn inspection images into structured information. The beginner journey starts with images as arrays, then moves through convolutional neural networks, transfer learning, evaluation, detection, segmentation, and finally annotated reporting. Each step matters because a model is only useful when its outputs match the engineering question and can be communicated clearly to a human reviewer.

The most important lesson is that the technical and engineering views must stay connected. A model score is not the end of the process. The end of the process is a defensible engineering judgment supported by visual evidence, class-sensitive evaluation, and honest communication of confidence and limitations.

References

- [1] OpenCV documentation, "Image Processing in OpenCV." [Online]. Available: <https://docs.opencv.org/>. Accessed: 2026-04-12.
- [2] Stanford University, "CS230 Cheatsheet: Convolutional Neural Networks." [Online]. Available: <https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-convolutional-neural-networks/>. Accessed: 2026-04-12.
- [3] TensorFlow, "Transfer learning and fine-tuning." TensorFlow Tutorials. [Online]. Available: https://www.tensorflow.org/tutorials/images/transfer_learning. Accessed: 2026-04-12.
- [4] TensorFlow, "Data augmentation." TensorFlow Tutorials. [Online]. Available: https://www.tensorflow.org/tutorials/images/data_augmentation. Accessed: 2026-04-12.
- [5] Massachusetts Institute of Technology, "VisionBook: Object Recognition." [Online]. Available: https://visionbook.mit.edu/object_recognition_v3.html. Accessed: 2026-04-12.
- [6] scikit-learn developers, "Confusion matrix example." [Online]. Available: https://scikit-learn.org/stable/auto_examples/model_selection/plot_confusion_matrix.html. Accessed: 2026-04-12.
- [7] scikit-learn developers, "Precision-Recall example." [Online]. Available: https://scikit-learn.org/stable/auto_examples/model_selection/plot_precision_recall.html. Accessed: 2026-04-12.
- [8] Microsoft Learn, "Tutorial: Automated visual inspection using transfer learning - ML.NET." [Online]. Available: <https://learn.microsoft.com/en-us/dotnet/machine-learning/tutorials/image-classification-api-transfer-learning>. Accessed: 2026-04-12.
- [9] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. Cambridge, MA: MIT Press, 2016.
- [10] F. Chollet, Deep Learning with Python, 2nd ed. Shelter Island, NY: Manning, 2021.